Cache Your Prompt When It's Green — Carbon-Aware Caching for Large Language Model Serving

YUYANG TIAN, University of Waterloo, Canada
DESEN SUN, University of Waterloo, Canada
YI DING, Purdue University, United States
SIHANG LIU, University of Waterloo, Canada

As large language models (LLMs) become widely used, their environmental impact, especially the carbon emission problem, has attracted more attention. Prior studies focus on compute-related carbon emissions. In this paper, we find that storage is another key contributor. LLM caching, which saves and reuses KV caches for repeated context, reduces operational carbon by avoiding redundant computation. However, this benefit comes at the cost of embodied carbon from high-capacity, high-speed SSDs. As LLMs scale, the embodied carbon of storage grows significantly. To address this tradeoff, we present GreenCache, a carbon-aware cache management framework that dynamically derives resource allocation plans for LLM serving. GreenCache analyzes the tradeoff between carbon emissions and SLO satisfaction, reconfiguring the resource over time to maintain this balance under dynamic workloads. Evaluations from real traces demonstrate that GreenCache achieves an average carbon reduction of 15.1 % when serving Llama-3 70B in the FR grid, with reductions reaching up to 25.3 %, while staying within latency constraints for > 90 % of requests.

CCS Concepts: • **Computer systems organization**; • **Computing methodologies** → **Machine learning**; • **Social and professional topics** → **Sustainability**;

Additional Key Words and Phrases: Sustainability, Large language model, context caching, carbon emissions

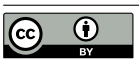
ACM Reference Format:
Yuyang Tian, Desen Sun, Yi Ding, and Sihang Liu. 2026. Cache Your Prompt When It's Green — Carbon-Aware Caching for Large Language Model Serving. *Proc. ACM Meas. Anal. Comput. Syst.* 10, 1, Article 5 (March 2026), 28 pages. https://doi.org/10.1145/3788087

1 Introduction

Large language models (LLMs) become widely adopted across applications—from dialogue systems [14, 24, 56] to healthcare [59] and developer tools [23]. Although LLMs demonstrate remarkable capabilities, they raise concerns about the environmental impact [16, 19, 37–39, 63, 67, 68, 70], particularly in terms of carbon emissions measured in carbon dioxide equivalent (CO_2e).

Carbon emissions in LLM serving systems come from two main sources: operational carbon and embodied carbon. Operational carbon emissions come from the electricity consumed by computing. The amount of carbon emitted per kilowatt-hour (kWh) depends on the grid's energy source mix, quantified by the *carbon intensity* (CI), measured in the unit of gCO_2e/kWh [49]. Renewable energy sources like solar and hydro have low carbon intensities, while non-renewable sources like coal and gas have high carbon intensities. Embodied carbon emissions come from the hardware

Authors' Contact Information: Yuyang Tian, yuyang.tian@uwaterloo.ca, University of Waterloo, Waterloo, ON, Canada; Desen Sun, desen.sun@uwaterloo.ca, University of Waterloo, Waterloo, ON, Canada; Yi Ding, yiding@purdue.edu, Purdue University, West Lafayette, IN, United States; Sihang Liu, sihangliu@uwaterloo.ca, University of Waterloo, Waterloo, ON, Canada.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2476-1249/2026/3-ART5
https://doi.org/10.1145/3788087

manufacturing process [26, 75]. Serving LLMs at scale is compute-intensive. It demands high power that will likely lead to high operational carbon emissions, and at the same time, requires high-performance hardware like high-end GPUs and machine learning accelerators that have high embodied carbon. For example, serving a single LLM prompt can emit over 20 times more carbon than a conventional web search query [25, 74].

To reduce the carbon emissions of LLM serving, prior work has focused primarily on optimizing the compute side (e.g., GPUs). For example, DynamoLLM [70] improves energy efficiency to lower operational emissions; GreenLLM [67, 68] and EcoServe [39] reuse older GPUs to lower embodied emissions. However, these efforts largely overlook another major contributor to carbon impact in LLM serving systems: storage.

While compute-related carbon emissions have been widely studied, the carbon emissions of storage remain an underexplored but critical component in sustainable LLM deployment. LLM tasks often involve a long context to provide specific information for better generation quality, such as long chat histories in multi-turn conversations and long documents in comprehension tasks. Processing these long inputs can require thousands of tokens before generation [42]. To address this, prior work [20, 21, 42, 77] has introduced context caching, which saves the KV cache of contexts in fast storage and reuses them when needed again. This reduces redundant computation, lowers latency, and decreases resource usage, ultimately reducing operational carbon emissions.

Despite the benefits, context caching requires high-capacity and high-speed storage such as SSDs, which come with embodied carbon emissions. Prior work [7, 15, 33, 51, 52, 65, 71] has shown that SSDs contribute to over 75 % of datacenter servers' total embodied carbon emissions [15, 71]. This means that caching does not come for free, but can significantly increase the overall carbon emissions of LLM serving. In a cloud environment, storage is provisioned on demand and can scale. Therefore, the embodied carbon from service is counted by the storage size and its time of usage, like prior studies on embodied carbon of hardware [26, 27]. However, there is no effective characterization or optimization strategy that optimizes storage carbon for LLM serving.

In this work, we present GreenCache, a carbon-aware caching framework to mitigate storage-related carbon emissions from caching in LLM serving systems. Balancing the tradeoff between increased embodied carbon and reduced operational carbon makes identifying the optimal caching configuration particularly challenging. To guide cache decisions, we characterize LLM serving caching behavior and make two key observations. First, higher loads amplify both latency reduction and carbon savings from caching, as cache hits substantially shorten the prefill phase. Second, carbon emission savings are strongly influenced by the carbon intensity (CI). When the carbon intensity becomes high, operational carbon from computation dominates; conversely, when it is low, embodied carbon from cache storage (SSD in this study) becomes the primary contributor. Both the load and CI fluctuate in a realistic environment, as load varies with user activity [70], and CI changes with real-time energy sources [49]. These insights reveal that the optimal cache configuration is jointly determined by these dynamic factors — the load, which governs operational carbon savings through latency reduction, and the carbon intensity, which affects the net carbon savings. Accordingly, GreenCache adapts to both load and CI conditions to continuously optimize cache configuration for optimal carbon efficiency.

GreenCache first profiles the performance and power under various workloads and cache sizes. In practice, GreenCache predicts both carbon intensity and workload based on historical values, enabling adaptive cache reconfiguration that anticipates future dynamics up to 24 hours in advance to preserve enough time to allow sufficient warm-up. However, finding the most carbon-efficient cache size is not sufficient. LLM serving applications are typically required to meet Service Level Objectives (SLOs) for both Time to First Token (TTFT) and Time to Produce Output Token (TPOT), with a target attainment rate (e.g., 90 %) to ensure service quality [62, 70, 81]. To ensure SLO

attainment while achieving carbon efficiency, we model this optimization as an Integer Linear Programming (ILP) problem. GreenCache integrates an ILP solver that takes the predicted carbon intensity and load, current TTFT and TPOT, and the SLOs as input, and determines the most carbon-efficient cache configuration that meets the SLO target, effectively balancing the operational carbon savings (from reduced computation) and embodied carbon emissions (from storage).

In addition, we introduce a new cache replacement policy for GreenCache, as conventional cache replacement policies such as Least Recently Used (LRU) do not optimize for carbon emissions. We find that longer reused context (i.e., more tokens upon a cache hit) yields more computation savings, reducing the operational carbon. On the other hand, the cache entry size varies, leading to different embodied carbon overheads. Therefore, we design a new policy, Least Carbon Savings (LCS), that incorporates not only cache access frequency and recency, but also the embodied carbon costs and operational carbon savings from reused context.

We implement GreenCache on top of an open-source context caching system, LMCache [45]. GreenCache leverages input SLO constraints and real-time carbon intensity to minimize total carbon emissions in serving LLMs while meeting the performance SLO attainment goal. We adapt two LLMs on top of the GreenCache framework: Llama-3 70B and 8B models [53], and evaluate on two tasks: multi-turn conversation using a ShareGPT dataset [30] and document reading comprehension using the TriviaQA dataset [32]. For a realistic evaluation, we take 24-hour LLM request rates from the Azure dataset [3] and carbon intensities from the CarbonCast dataset [49]. Experiments are conducted on a system with 4× NVIDIA L40 GPUs and configurable SSD storage of up to 16 TB.

The contributions of this paper are as follows:

- To the best of our knowledge, this is the first work that systematically studies the embodied carbon emissions of storage due to caching in LLM serving systems.
- We design and implement a caching framework for LLM serving that dynamically reconfigures cache size in response to fluctuating CIs and workloads, while meeting the SLO attainment goal.
- Our evaluation based on realistic request rate and CI traces demonstrates that GreenCache reduces average carbon emissions by 15.1 % in low-CI grids such as FR, and can still achieve up to a 6.91 % reduction in higher-CI grids like CISO when serving the Llama-3 70B model.
- The source code of this work is available at https://greencache.persistentmemory.org.

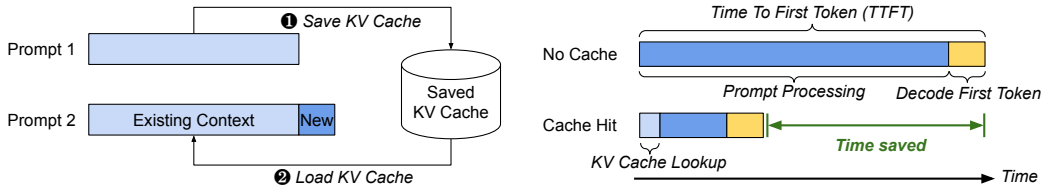
2 Background

In this section, we first introduce the LLM serving process. We then discuss the cache mechanism for LLM serving for performance optimization. Finally, we discuss carbon accounting methods for operational and embodied emissions.

2.1 LLM Serving Process

LLM serving, typically based on the Transformer architecture, operates by predicting the output in a sequence given a prompt. LLM serving has two phases: *prefill* and *decode*. The prefill phase (or prompt phase) processes the input prompt and generates the first output token. Its performance metric is measured by *Time To First Token (TTFT)* — the time from receiving the prompt to producing the first token. The decode phase (or token generation phase) generates tokens one by one in an autoregressive manner. Its performance metric is measured by *Time Per Output Token (TPOT)* — the time between each output token.

While attention operations are computationally intensive, most operations with Key and Value remain the same across decode iterations, providing an opportunity to store and reuse these tokens, referred to as the *KV cache* [34]. With *KV cache*, the decode phase becomes memory-intensive rather than compute-intensive, and increasing the batch size can significantly improve throughput.



(a) Caching procedure. (b) Timeline of no cache and a cache hit.

Fig. 1. Illustration of caching for LLM serving.

In LLM serving systems, batching is a key technique for maximizing throughput [34, 78, 81]. Naively batching all existing requests and disallowing batch enlargement mid-processing results in suboptimal throughput. To address this, recent work introduces continuous batching [78], which allows new requests to be inserted during the decode phase, significantly improving GPU efficiency.

2.2 Caching for LLM Serving

To further reduce the heavy computation in the prefill phase, recent work has introduced caching techniques [11, 20, 21, 34, 42, 77]. These studies indicate that many requests share overlapping tokens. For example, in multi-turn conversations, later turns often include the entire chat history. In reading comprehension, multiple questions may ask about the same document. Caching enables reusing the KV cache from previous requests, avoiding redundant computation for repeated context and trading off extra storage for better performance.

Figure 1a illustrates a system that reuses the KV cache of the context. Step ❶: upon a new request Prompt 1, the system processes it and saves its KV cache to storage. Step ❷: when the next request Prompt 2 arrives, the system looks up the cache for the existing context. When hit, the system stitches the KV cache of the cached tokens and the new tokens, and processes them together. Figure 1b depicts the timeline with or without cache. Due to the intensive attention computation, KV cache lookup incurs a much lower overhead than processing. For example, the average TTFT in executing prompts in the ShareGPT dataset [30] using the Llama-3 70B model on 4× L40 GPUs is 1.7 seconds, but loading the KV cache of previous context only takes 0.03 seconds on average. Therefore, cache reduces TTFT significantly by eliminating redundant processing in the prefill phase. Even though caching does not reduce computation in the decode phase, it reduces the waiting time for the decode phase in continuous batching, improving the decode latency. Despite the performance gains, an effective caching system fundamentally relies on the storage backend to maintain a high volume of KV cache data that can be accessed at high speed. For example, caching a 1000-token context for 1 million prompts of a Llama-3 70B model takes more than 300 TB [44].

2.3 Carbon Emission Accounting

LLM serving is highly compute-intensive and results in significant environmental impact. According to the carbon modeling tool from prior work [19, 26, 36–38, 55, 67, 68], the total carbon emissions of serving LLMs on a hardware platform are the sum of the operational (C_o) and embodied (C_e) carbon emissions. As the cloud dynamically allocates resources, a user's embodied carbon emissions are attributed to the duration for which the computing platform is allocated. Therefore, the embodied carbon emissions are proportional to the execution time T, amortized over the platform's lifetime (LT). For example, the typical lifetime of hardware components is around 5 years [26, 57], which

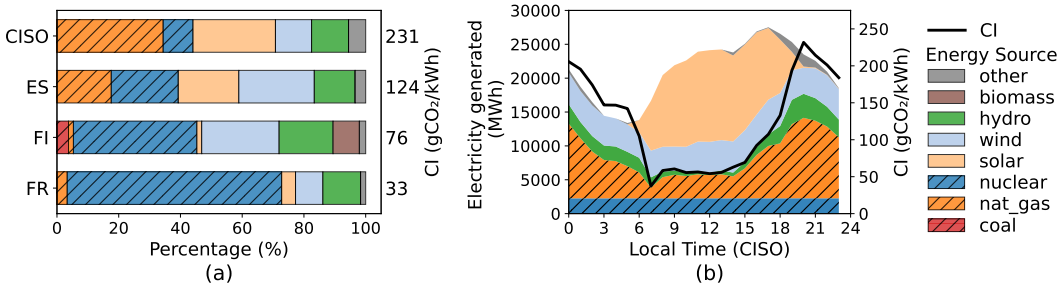


Fig. 2. (a) Average carbon intensity (CI) and energy sources of four grids in 2024 [18]. (b) CI variation due to energy sources of the CISO grid on July 6, 2022 [49].

Table 1. Embodied carbon of major server components.

Component	Type	Embodied Carbon
CPU	AMD 7453	9.3 $kgCO_2e$ [26]
GPU	4 × NVIDIA L40	106.4 $kgCO_2e$ [26, 55]
Memory	512 GB DDR4	30.8 $kgCO_2e$ [26]
Storage	(Up to) 16 TB SSD	(Up to) 480 $kgCO_2e$ [26]

we follow in the rest of this paper unless specified. We calculate the total carbon emissions as

$$C = C_o + \frac{T}{LT} C_e. \tag{1}$$

The *operational carbon* (C_o) is computed as the product of the energy E consumed by running LLMs on the target hardware and the carbon intensity (CI) during the execution

$$C_o = E \times \mathtt{CI}, \tag{2}$$

where CI measures the amount of carbon dioxide equivalent emitted per unit of electricity generation (i.e., gCO_2e/kWh). A lower carbon intensity value indicates a higher proportion of renewable energy used in electricity generation from the power grid. Figure 2a shows four grids: FR (France), FI (Finland), ES (Spain), and CISO (California, USA), which differ in their compositions of energy sources. Figure 2b shows the electricity sources vary in the CISO grid throughout a day.

The *embodied carbon* (C_e) from hardware comes primarily from the semiconductor manufacturing process. In this work, we focus on the main hardware components that contribute to the majority of embodied carbon for LLM serving, including GPUs, CPUs, memory, and storage. Table 1 lists the hardware components and their embodied carbon, which is modeled based on their processor chip area and memory capacity [26]. We calculate the total embodied carbon emissions of the system running LLM service as

$$C_e = \sum_{\text{comp} \in \text{Platform}} C_{e,\text{comp}} = C_{e,\text{GPU}} + C_{e,\text{CPU}} + C_{e,\text{Mem}} + C_{e,\text{SSD}}. \tag{3}$$

While prior studies have examined the embodied carbon of compute components like CPUs and GPUs, the impact of storage, particularly SSDs, for LLM serving remains underexplored, despite evidence that it contributes significantly to embodied emissions [7, 15, 36, 51, 65, 71]. In our platform, the embodied carbon from SSDs consumes 76.6 % of the total embodied carbon of the server, similar to the over 75 % fraction reported by a prior study [71].

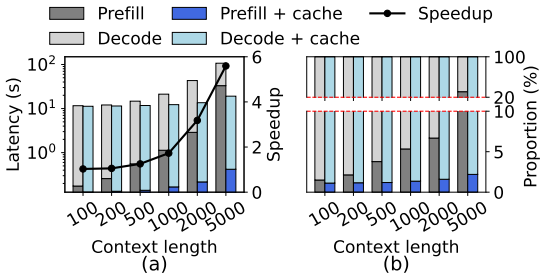


Fig. 3. (a) Latency and speedup from caching under different context lengths. (b) Latency breakdown.

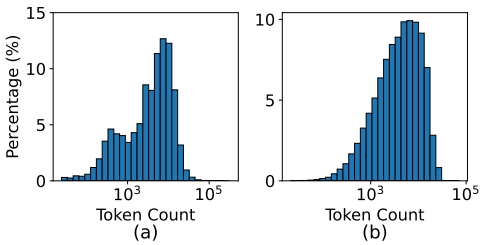


Fig. 4. Distribution of context length in (a) ShareGPT [30] and (b) TriviaQA [32].

3 Performance and Carbon Emission of LLM Caching

In this section, we analyze the performance and carbon emissions of caching in LLM serving and summarize the observations as takeaways. We illustrate the results using the Llama 3 70B model [53] and the ShareGPT dataset [30], evaluated on the server described in Table 1. We use LMCache [45] as the caching system, which saves context on 16 TB SSDs. The cache has been initialized with 200k prompts, and we record the performance and carbon emissions of 500 prompts. Details of our evaluation methodology are provided in Section 6.1.

3.1 Performance of LLM Caching

We first study the performance of context caching in LLM serving by focusing on two factors: context length and request rate. Since the prefill phase accounts for only a small fraction of the total latency, we present all latency results in log scale to better visualize the prefill latency.

3.1.1 Impact of Context Length. Caching reduces the computation of the existing context. Therefore, we first evaluate its benefits under different context lengths. Figure 3a shows the prefill and decode latency breakdown and speedup from caching under variable context lengths. The speedup increases as the cached context gets longer. Furthermore, caching brings more improvements for the prefill phase than the decode phase, as the fraction in Figure 3b shows. This is because prefill directly benefits from cached context, but decode indirectly benefits from reduced waiting time.

We further analyze the context length in two common LLM tasks where caching can be useful. First, we study the context length of a multi-turn conversation task using a ShareGPT dataset [30, 66]. Each conversation contains a number of turns between the user and the LLM chatbot. We observe that the context length varies, as Figure 4 shows that 77.2 % of prompts have over 1000 tokens from prior conversation turns. Second, we study another task, document reading comprehension, using the TriviaQA dataset [32]. Unlike multi-turn conversation, its context is the whole document that corresponds to the user's question, with an average context length of 5880 tokens. Therefore, the majority of requests benefit from context caching.

Takeaway 1

Longer context lengths yield more benefits from caching, as more redundant prefill computation can be eliminated. However, context length varies in tasks and prompts, leading to variable benefits. Therefore, longer contexts should be prioritized in caching.

3.1.2 Impact of Request Rate. The request rate is a key factor that impacts the performance benefits from caching. We conduct an experiment to evaluate the latency under different request rates,

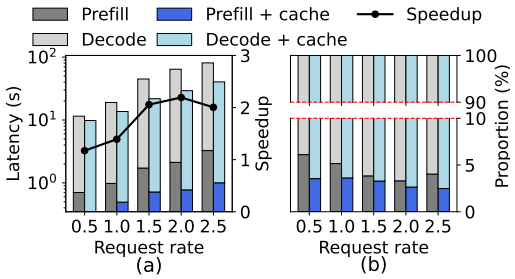


Fig. 5. (a) Latency of prefill and decode under different request rates, and the speedup from caching. (b) Fraction of prefill and decode latency.

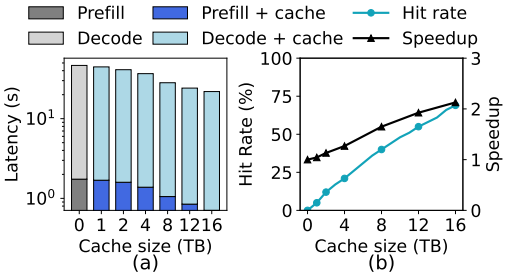


Fig. 6. (a) Latency and speedup from caching under different cache sizes. (b) Cache hit rate.

as Figure 5 shows. As the request rate becomes higher, the average latency reduction of prefill increases. At the same time, the average decoding latency also reduces and benefits more when the request rate is higher. This is because reduced prefill latency also saves waiting time for decoding, as discussed in Section 2.2.

Takeaway 2

Higher request rates benefit more from caching, as the computation reduction by cached context is more prominent when the request rate is high. Therefore, more context should be cached when the system is under high load.

3.1.3 Impact of Cache Size. The cache size is another key factor for performance benefits. We set a fixed request rate of 1.5 prompts/s and vary the cache sizes from 1 to 16 TB. Then, we compare the performance with the no-cache configuration. Figure 6 shows that larger cache sizes yield higher speedup and lower latency, as more prompts hit and reuse the KV cache of the cached context.

Takeaway 3

Larger cache sizes improve performance by increasing hit rates, allowing more prompts to reuse KV caches. However, the benefit may not always scale linearly, as the hit rate improvement slows down once the cache reaches a certain size.

3.2 Carbon Emissions of LLM Caching

In this subsection, we model and characterize the carbon emissions of LLM caching systems.

3.2.1 Cache Carbon Modeling. Section 2.3 introduces the standard carbon modeling method for computing systems. Building upon it, we introduce a carbon modeling approach for LLM caching. In this work, we use SSD, the most commonly used storage in LLM serving systems [20, 21, 42, 45, 77], as the cache hardware.[1] In a cloud environment, storage is provisioned on demand and can be flexibly resized. In LLM context caching, the cache size reflects the number of cached tokens, as the service acquires more storage when the number of cached tokens increases and releases storage when the demand for cached tokens decreases. We model the cache's embodied carbon as proportional to the current allocated storage size S_{Alloc}, reflecting a cloud scenario where only the reserved storage contributes to embodied carbon, rather than the entire device. Let $C^{\text{Unit}}_{\text{e,SSD}}$ denote the embodied carbon per unit of storage (e.g., $kgCO_2e$ per TB). The embodied carbon attributed to

[1]The same carbon modeling also applies to other caching mediums, such as DRAM, CXL-attached memory, and HDD.

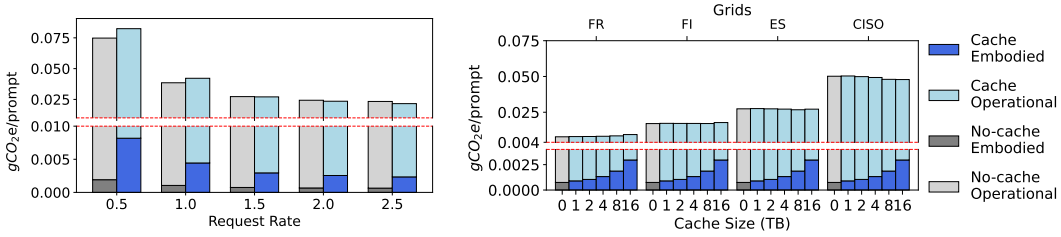


Fig. 7. Carbon emissions per request (a) in the ES grid under different request rates, and (b) under different cache sizes and grids (based on grid average CI in Figure 2a).

the allocated SSD is then scaled by the fraction of its lifetime during which the storage is occupied. In our evaluation system, we limit the SSD size to 16 TB, as shown in Table 1. Accordingly, we define the carbon emissions of the cache as follows:

$$C_{\text{e,Cache}} = \text{S}_{\text{Alloc}} \times \frac{\text{T}}{\text{LT}} C^{\text{Unit}}_{\text{e,SSD}}. \tag{4}$$

For operational carbon emissions, we first measure the power of every server component when serving LLM and the prompt execution latency to obtain energy consumption. Then, we multiply it by the carbon intensity of the grid, as introduced in Equation (2). Together, the total carbon emissions can be calculated using Equation (1), expressed as:

$$C = E \times \text{CI} + \frac{\text{T}}{\text{LT}} \left(C_{\text{e,Cache}} + C_{\text{e,Others}} \right). \tag{5}$$

This equation indicates a tradeoff between embodied and operational carbon emissions. Caching improves LLM serving performance by lowering the execution time and energy consumption, which reduces operational carbon emissions (the $E \times \text{CI}$ component). However, caching that uses extra storage incurs embodied carbon (the $C_{\text{e,Cache}}$ component). Next, we examine how different serving conditions impact the carbon emissions of LLM caching.

3.2.2 Carbon Emissions of LLM Caching. We first measure the caching system's carbon emissions under varying request rates. Figure 7a demonstrates that the average per-prompt carbon emissions vary with the request rate. All carbon emissions are calculated with ES grid's carbon intensity. At low request rates, caching incurs higher emissions due to the embodied carbon of SSDs. As the request rate increases, operational carbon savings from caching become dominant, while the relative impact of embodied carbon diminishes.

Takeaway 4

The carbon emission savings depend on the request rate. At higher loads, caching yields greater carbon reductions by effectively mitigating the elevated operational carbon.

We next study the impact of carbon intensity. Under a request rate of 1.5 prompts/s, we calculate the total carbon emission savings using the average carbon intensity (CI) from 12 grids [18], as shown in Figure 8a. Grids with a higher CI (on the right side) benefit more from caching. For instance, a 16 TB cache reduces carbon emissions by 7.5 % in MISO, where the CI reaches 485 gCO_2e/kWh. In contrast, low-CI grids (on the left side) benefit less or even incur higher emissions from caching. For example, in FR, where the CI is only 33 gCO_2e/kWh, the same cache increases carbon emissions by 16.5 %. As CI fluctuates within the same grid due to changing energy sources, we next evaluate

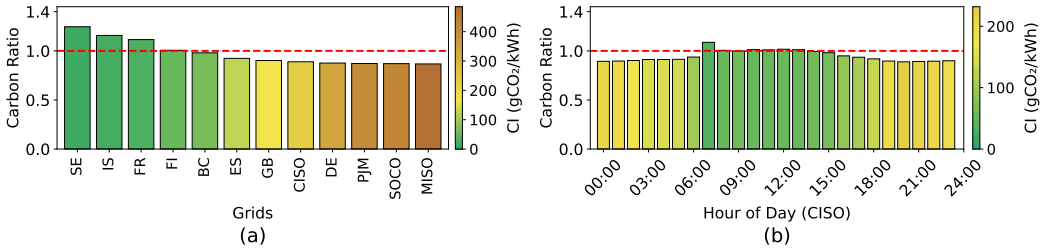


Fig. 8. Carbon emission savings from caching in 12 grids. A ratio < 1 indicates carbon emission reduction.

the carbon emission savings of a 16 TB cache over a day in the CISO grid, as shown in Figure 8b. The CI drops to its daily minimum of 37 gCO_2e/kWh at 7 AM when the grid is mainly powered by renewable energy sources (as shown in Figure 2b), making caching not carbon-efficient. Conversely, at 8 PM, CI peaks at 232 gCO_2e/kWh, and caching achieves its maximum carbon reduction.

The shift in carbon emission savings results from changes in the primary carbon contributor within the LLM serving system. Consequently, when CI is high, a larger cache can save more computations and significantly reduce operational carbon, reducing the total carbon per prompt. Conversely, at low CI, embodied carbon dominates the total carbon emissions. Thus, increasing the cache size may negatively impact the carbon emissions.

To better understand this tradeoff between embodied carbon and operational carbon, we further evaluate the carbon emission savings under cache sizes ranging from 1 to 16 TB with the average CI of four grids in Figure 2, as shown in Figure 7b. In all grids, the fraction of embodied carbon rises with cache size, while operational carbon reduction is more significant in high-CI grids. These results indicate that a fixed cache configuration is suboptimal under variable CIs.

Takeaway 5

Carbon emissions depend on carbon intensities and cache sizes. A higher carbon intensity can lead to lower carbon emissions with a larger cache, as the cache saves operational carbon due to computation reduction. When the carbon intensity is low, a large cache can increase the carbon emissions due to its embodied carbon. Because the carbon intensity is highly dynamic, both across grids and within a grid, an adaptive method is needed to achieve optimal carbon savings.

4 High-level Ideas of GreenCache

GreenCache is a carbon-aware caching framework for LLM serving by making tradeoffs between the operational carbon reduction from caching and its extra embodied carbon. Next, we introduce two main ideas that enable adaptive caching and guarantee SLO attainment behind it.

4.1 Carbon-aware Adaptive Caching

Takeaways 4 and 5 reveal that the carbon savings from caching depend on both request rate and carbon intensity. These two factors affect the choice of cache size configuration dynamically, as they depend on the task, time, and the grid. Therefore, the first challenge is to find the optimal cache size given the dynamic factors.

GreenCache resizes the cache according to the request rate and carbon intensity during runtime. The high-level idea is to first profile the LLM serving system's performance and power under different combinations of request rates and carbon intensity levels. When serving prompts, GreenCache can find the optimal configuration of cache size that leads to the lowest per-prompt carbon

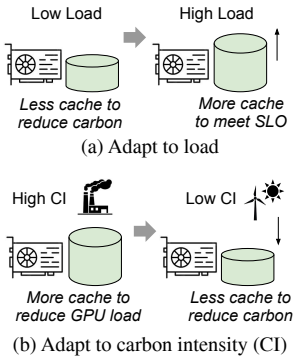


Fig. 9. Adaptive caching.

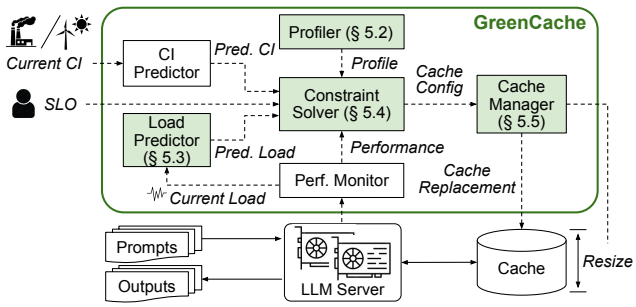


Fig. 10. System overview of GreenCache. Components in green are newly designed by this work.

emissions. However, one challenge is that cache resizing, especially enlarging the cache, takes time to warm up, and thus the performance highly depends on the future conditions. To overcome this challenge, GreenCache predicts the request rate and carbon intensity based on historical data, enabling optimal decision-making that considers not only the current condition but also future trends. Figure 9a illustrates an example where GreenCache adaptively increases the cache size when the predicted load (i.e., request rate) increases to better reduce the operational carbon. Figure 9b shows another example where GreenCache adaptively shrinks the cache size when the carbon intensity is predicted to reduce, to save the embodied carbon of cache storage.

4.2 Performance SLO Attainment

Only optimizing for carbon emissions can also negatively impact performance. As Takeaway 3 has shown, a small cache can lead to the minimum carbon emissions under low carbon intensity, but significantly degrade performance. In LLM serving systems, Service Level Objectives (SLOs) typically define latency constraints for TTFT and TPOT, with the goal of ensuring that a high percentage of requests (e.g., 90 %) adhere to these thresholds. Cache configurations that fail to achieve the required SLO attainment are not acceptable, even if they minimize carbon emissions. Thus, the second challenge lies in minimizing carbon emissions while ensuring SLO attainment.

GreenCache uses a constraint solver to find such a cache configuration. We formulate the constrained carbon emission optimization as an Integer Linear Programming (ILP) problem. The solver has an objective function that minimizes the carbon emissions based on the profile discussed in the first high-level idea. At the same time, it enforces constraints to ensure SLO attainment. In scenarios where a small cache is preferred to minimize carbon but can miss most SLOs, the solver guides GreenCache to choose a larger cache that achieves targeted SLO compliance.

5 GreenCache Framework

We first present an overview of GreenCache and then describe its components.

5.1 System Overview

We design and implement GreenCache, a carbon-aware caching framework for LLM serving. Figure 10 presents the system overview of GreenCache. GreenCache consists of six major components. First, a *profiler* periodically analyzes the LLM task to model the relationship between cache size, load, performance, and power consumption (details in Section 5.2). Second, a *performance monitor* tracks the TTFT and TPOT of LLM serving and the current load (i.e., request rate). Third, a

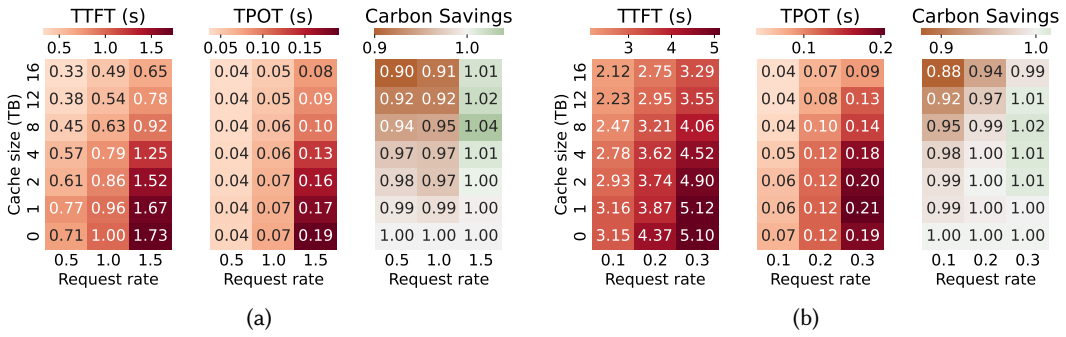


Fig. 11. Profiling results of TTFT and TPOT (lower is better), and carbon savings over no-cache (higher is better) when serving Llama-3 70B in the ES grid for (a) multi-turn conversation, using a ShareGPT dataset [30] and (b) document comprehension using a TriviaQA dataset [32] (with skewness of $\alpha = 0.4$).

combination of *CI predictor* and *load predictor* forecasts the CI and request rate based on historical values. Specifically, we use the state-of-the-art CI predictor, EnsembleCI, [76] for CI prediction (methodology details in Section 6.1), and design a load predictor (details in Section 5.3). A *constraint solver* takes the SLO, predicted CI, performance, and power profile, predicted load, and current performance as input, and identifies the cache configuration that minimizes carbon emissions while attaining SLO (details in Section 5.4). Finally, a *cache manager* resizes the cache according to the cache configuration. It also incorporates a carbon-aware replacement policy that replaces cache entries with the Least Carbon Savings (LCS) (details in Section 5.5). We build GreenCache on top of the LLM caching system, LMCache [45]. GreenCache can be flexibly adapted to different LLMs.

We next introduce the new components in the GreenCache framework, marked green in Figure 10.

5.2 Cache Performance Profiler

The cache performance profiler analyzes performance and power under different request rates and cache sizes for each LLM task. The profiler samples a number of prompts and evaluates them on an initialized cache filled to maximum capacity. The profiler consists of two components for evaluation, a monitoring tool and a carbon emission calculator, to profile each LLM task.

Performance and Power Analysis. The profiler sweeps cache sizes and request rates. Cache size values are defined by the cache configurations, while request rates are varied up to the maximum level the system can support before violating SLOs. For each combination of cache size and request rate, it records the TTFT and TPOT of every prompt. In parallel, it records the power consumption of key server components. The CPU power is measured using a RAPL tool [8] and GPU power is measured using pyNVML [61] every 1 ms. For other relatively low-power components, like SSD and DRAM, that do not provide direct measurement interfaces, the monitoring tool follows the typical power in their specifications [54, 64]. In real-world LLM serving scenarios, profiling is performed periodically to adapt to dynamic changes in workload characteristics, such as distribution of request length and access patterns, as seen in prior work [40].

Carbon Calculation. The carbon calculator follows the approach in Sections 2.3 and 3.2.1. It first computes the total energy consumption using the timing and latency measurements and then derives the operational carbon based on the grid's current carbon intensity, as defined in Equation (2). Next, it estimates the embodied carbon of the platform using Equation (3). Specifically, it calculates the embodied carbon of the SSD for caching via Equation (4). Finally, the total carbon emissions are obtained by summing the operational and embodied components, following Equation (5).

Table 2. Key notations in the ILP problem.

t	Timestamp during the LLM execution	$p_i^{j_t}$	Average power of request i at request rate j_t
$\texttt{CI}_t$	Carbon intensity (CI) at time t	$\texttt{TTFT}_i^{j_t}$	TTFT for request i at request rate j_t
j_t	Request rate at time t	$S_i^{j_t}$	Cache size of request i at request rate j_t
N	Total number of requests in an LLM service	$\texttt{LT}_{\text{comp}}$	Lifetime of comp $\in$ {SSD, GPU, CPU, Mem}

In this work, we adapt two LLM tasks to GreenCache, multi-turn conversation and document reading comprehension, which have also been studied in prior works [10, 17, 20, 31, 41, 42, 46, 81]. For each task, we sample 500 prompts from the dataset and execute them after cache warmup. We demonstrate the profiling results of both tasks (ShareGPT dataset [66] for multi-turn conversation and TriviaQA [32] for document reading comprehension) using a Llama-3 70B model [53] on the platform described in Table 1. As the profile targets GreenCache, the cache uses our carbon-aware policy (details in Section 5.5). More experiment methodology details are described in Section 6.1.

Figure 11a shows heatmaps of the average TTFT and TPOT, and carbon savings from caching, under variable request rates (x-axis) and cache sizes (y-axis), for the multi-turn conversation task. We profile TTFT and TPOT under various cache sizes (y-axis). We also profile the carbon emission savings based on the CI of the ES grid — defined as the ratio between the no-cache and cached configurations — where a ratio greater than 1 indicates benefits. The TTFT and TPOT in the profile are consistent with Takeaways 2 and 3. Overall, larger caches improve the latency, and the benefit is more prominent for higher request rates. The trend of carbon savings is also aligned with Takeaway 4. Larger caches may not bring carbon emission savings, especially when the request rate is low. When the rate is high, larger caches yield more savings. Figure 11b shows the heatmaps of the document reading comprehension task with skewness of $\alpha = 0.4$ (i.e., 10 % of documents are accessed by ~25 % of prompts). The latency and carbon savings trends are consistent with the multi-turn conversation task. However, the longer document contexts result in a TTFT, which lowers the maximum request rate in this profile. These profiles will be used by a constraint solver (Section 5.4) to select the optimal cache size configuration.

5.3 Load Predictor

Prior work from Microsoft has shown that LLM prompts follow a similar pattern in a day [70], where the rate mostly depends on the time of day. We design a lightweight predictor, based on the Seasonal Autoregressive Integrated Moving Average (SARIMA) model, using *pmdarima* [69]. It captures both daily periodicity and short-term autocorrelation in the given load. The SARIMA parameters are estimated from historical rate traces using maximum likelihood fitting. We also use *pmdarima* for parameter auto-tuning. We use a hold-out evaluation, where it takes the most recent three consecutive days of data as input and predicts 24 hours ahead. Moreover, the load predictor performs online step-ahead prediction during runtime — every hour, the model incorporates the most recent load before forecasting future hours to adapt to rate fluctuations. This predictor incurs minimum computation and runs on separate CPU cores, without interfering with the LLM workload.

5.4 Constraint Solver

We first describe the formulation of the optimization problem. Then, we discuss our assumptions and potential error sources that can affect the decisions. Finally, we describe the solver's implementation.

5.4.1 Problem Formulation. GreenCache minimizes total carbon emissions by selecting the optimal cache size while meeting the SLO attainment goal. The total carbon emissions include both

operational carbon and embodied carbon emissions, where embodied emissions include storage (SSD) and non-storage (GPU, CPU, and memory).

The carbon optimization focuses on the prefill phase, where cache hits directly reduce computation by eliminating redundant processing for existing contexts. While cache size does not directly reduce auto-regressive token generation computations in the decode phase, it effectively reduces decode latency by accelerating the prefill phase, mitigating decode delays for the concurrently scheduled requests within a continuous batching system. For this reason, as discussed in Section 2.2, caching influences only the waiting time rather than the total computation of the decode phase.

We formulate this as an Integer Linear Programming (ILP) problem. The key notations are summarized in Table 2. An LLM service handles a total of N requests. At each time t, the LLM serving system experiences a request rate of j_t. Each request i has power consumption $p_i^{j_t}$, time-to-first-token latency $\mathrm{TTFT}_i^{j_t}$, where the request rate of a future time t is predicted using our load predictor. Given the predicted carbon intensity $\mathtt{CI}_t$ of a future time t, the operational carbon emissions for the prefill phase can be calculated by multiplying power, TTFT, and CI. Because of adaptive caching, the calculation of SSD embodied carbon emissions is different from other hardware components, as shown in Equation (4). It depends on both the cache size $S_i^{j_t}$ and execution time $\mathrm{TTFT}_{\mathrm{t}}^{\mathrm{j_t}}$. We calculate the embodied carbon of the SSD capacity allocated to the LLM service.

Then, given the execution time, we calculate the fraction of time a prompt utilizes the SSD's lifetime. Equation (6) summarizes the objective function and constraints for the ILP problem. For an LLM service with N requests in total with request rate j_t at time t, GreenCache minimizes total carbon emissions while meeting TTFT and TPOT SLOs as follows:

$$\underset{S_i^{j_t}}{\operatorname{argmin}} \sum_{i=1}^{N} \Bigg(\overbrace{p_i^{j_t} \mathrm{TTFT}_i^{j_t} \mathtt{CI}_t}^{\textbf{Operational carbon}} + \overbrace{\frac{\mathrm{TTFT}_i^{j_t}}{\mathtt{LT}_{SSD}} S_i^{j_t} C_{\mathrm{e,SSD}}^{\mathrm{Unit}}}^{\textbf{Cache embodied carbon}} + \overbrace{\sum_{\mathrm{comp} \in \mathrm{GPU,CPU,Mem}} \frac{t_{\mathrm{TTFT,i}}^{j_t}}{\mathtt{LT}_{\mathrm{comp}}} C_{\mathrm{e,comp}}}^{\textbf{Other embodied carbon}} \Bigg)$$

$$s.t. \quad \sum_{i=1}^{N} z_{\mathrm{TTFT,i}} \geq \rho N \bigwedge \sum_{i=1}^{N} z_{TPOT,i} \geq \rho N \,, \tag{6}$$

where $z_{\mathrm{TTFT,i}}, z_{\mathrm{TPOT,i}} \in \{0, 1\}$ are binary variables such that $z_{TTFT,i}$=1 if a request i meets the TTFT constraint, and $z_{\mathrm{TPOT,i}}$=1 means request i meets the TPOT constraint. ρ specifies the required fraction of prompts that meet the requirements. We set it as 0.9, meaning at least 90 % of requests must satisfy both TTFT and TPOT latency requirements, corresponding to the targeted SLO attainment.

5.4.2 Assumptions and Error Analysis. To formulate the optimization problem efficiently, we make three key assumptions. First, we constrain the cache size variable S_t to a discrete set of integers, which aligns with the granularity of cache sizes in the cloud. This constraint disallows the selection of fine-grained intermediate values. Second, we assume that the carbon intensity ($\mathtt{CI}_t$) remains constant within each decision interval (1 hour due to the granularity of the CI dataset [49]). Third, we assume that the profiled metrics (e.g., TTFT) reflect the system's performance under stable conditions. In particular, we collect the offline profiling data after a cache warm-up period using our LCS replacement policy. Therefore, the dependencies between cache sizes, cache hit rate, and the resultant latency are implicitly captured within the profiled results. Satisfying the SLO constraint requires exploring a combinatorial search space, scaling as $O(2^T)$ in the worst case, where T denotes the count of discrete optimization points defined by the granularity of timestamps. We further prove that this optimization problem is NP-hard via a reduction from the 0–1 Knapsack problem in Section A. However, the specific problem in GreenCache is computationally tractable due to the constrained decision space. Unlike the general Knapsack problem, our formulation involves

a limited set of discrete cache size candidates (1 TB granularity) and a finite time interval. These practical assumptions effectively prune the search space. We accept the resulting "rounding loss" as a necessary tradeoff to maintain the computational efficiency required for online decision-making.

In addition, we acknowledge three types of errors affecting theoretical optimality. First, the load predictor may introduce errors compared to the ideal outcomes. Second, the CI predictor can also introduce errors. Third, profiling error arises from discrepancies between the profiling phase and actual execution. We evaluate these errors in Section 6.5 and demonstrate that prediction and profiling errors have a low impact on total carbon savings.

5.4.3 Solver Implementation. We use the PuLP optimization modeling library [60] with the COIN-OR CBC solver [12] as the backend to solve the ILP objective function of Equation (6). GreenCache takes the optimal cache configuration determined by the ILP solver and performs cache resizing every hour. The cache has an allocation granularity of 1 TB, with a maximum size of 16 TB. We evaluate the ILP execution time in Section 6.4 — 7.03 s per decision on average, a low overhead compared to the hourly cache resizing frequency.

5.5 Cache Manager and Replacement Policy

The cache controller takes the cache size configuration from the constraint solver and manages the cache. To enlarge the cache, the cache controller allocates more SSD space from the cloud. When the cache shrinks, the cache controller evicts existing cache entries with the lowest scores until the total size reaches the configuration. Then, the spare cache space will be released.

The replacement policy determines the score of cache entries. To optimize carbon efficiency, GreenCache introduces *Least Carbon Savings* (LCS), a new carbon-aware cache replacement policy that prioritizes entries based on their potential carbon impact. Unlike conventional cache replacement policies that focus on either access recency (e.g., LRU) or frequency (e.g., LFU), LCS minimizes carbon emissions by evicting cache entries that offer the least carbon savings. As discussed in Section 5.4, carbon emissions from caching depend on both cache size and hit rate. We design a scoring function guided by four key insights:

(i) Prioritize entries with more hit tokens, which yield higher operational carbon savings, as shown in Takeaway 1.
(ii) Favor frequently accessed entries, as they are more likely to produce cache hits and contribute to greater carbon savings.
(iii) Prefer smaller entries that consume less storage, reducing embodied carbon from SSD usage. Although a longer context reduces more operational carbon as concluded in Takeaway 1, it costs more embodied carbon, which introduces a tradeoff.
(iv) Consider recency of access, as recently used entries are more likely to be accessed again soon.

Together, we design the LCS score as follows

$$Score = \frac{\#Token \times \#Hit}{Size \times Age}. \tag{7}$$

$\#Token$ is the accumulated hit token number of this cache, representing the volume of reused context. A higher number indicates more operational carbon savings, supporting *Insight (i)*. $\#Hit$ is the number of cache hits. Frequent access increases the chance of carbon savings by avoiding recomputation, supporting *Insight (ii)*. $Size$ is the cache size of the entry. Dividing by cache size encourages keeping smaller entries that consume less embodied carbon, supporting *Insight (iii)*. Age is how long the cache has stayed in storage, where older entries are less likely to be reused. Penalizing age promotes the eviction of stale entries, supporting *Insight (iv)*.

As described in Section 5.2, we evaluate two LLM tasks. Overall, *Size* and *Age* in Equation (7) work in the same way for both tasks, but we specifically adapt *#Token* and *#Hit* fields to each task.
Task 1: Multi-turn conversation. Each turn reuses the KV cache of context from previous turns. We adapt Equation (7) to prioritize cache entries that contribute more to carbon savings:

$$Score = \frac{CurTurn \times \#AccuToken}{Size \times Age}, \tag{8}$$

where *CurTurn* encourages retaining cache entries deeper in the conversation, since later turns rely more heavily on prior context. *#AccuToken* reflects the total reused tokens across turns, directly tied to operational carbon savings.
Task 2: Document reading comprehension. A document is reused across multiple questions, similar to turns in a dialogue. We treat these questions as equivalent to conversation turns and adapt the score accordingly:

$$Score = \frac{\#Hit \times AccuDocLen}{Size \times Age}, \tag{9}$$

where *#Hit* tracks the number of times a document is reused across questions, and *AccuDocLen* measures the total reused document length, capturing the operational carbon savings.

6 Evaluation

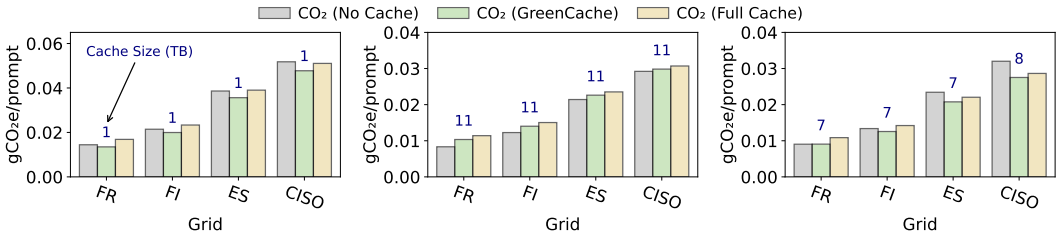
6.1 Methodology

Experiment setup. We evaluate two models, Llama-3 70B and 8B [53]. The 70B model runs on 4× NVIDIA L40 GPUs using the platform in Table 1, and the 8B model reduces the GPUs to 2× L40 as the model is less demanding. Due to GPU memory constraints, the 70B model runs under INT8. The 8B model runs under the default BF16. Both models have a context window of 8k tokens. When the context goes beyond this limit, we truncate extra context like prior work [20, 42]. The platform maintains a maximum capacity of 16 TB of SSD. GreenCache provisions SSD at a granularity of 1 TB to the LLM caching system. The power measurement method follows Section 5.2. The caching system is built on top of LMCache [45], which includes vLLM [34] and continuous batching [78] optimizations. We also integrate our carbon-aware LCS replacement policy into LMCache. We use a maximum cache size of 16 TB for the 70B model and 8 TB for the 8B model. By default, GreenCache resizes the cache every hour. Section 6.6.1 evaluates different resizing frequencies.

Tasks and Datasets. We evaluate two datasets that correspond to the two LLM tasks commonly used by prior works [10, 17, 20, 31, 41, 42, 46, 81]. In both datasets, we use distinct sets of prompts for profiling and evaluation.

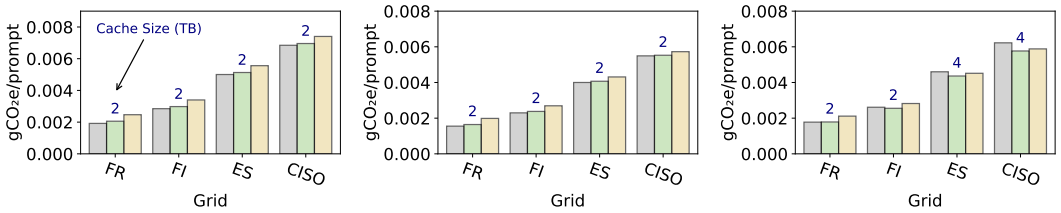
- **Multi-turn conversation based on ShareGPT [30, 66].** We randomly select a conversation every time and take its next conversation turn as the input prompt. The request follows a Poisson distribution like prior works [20, 21, 81]. By varying α, the request generator simulates different average request rates. We initialize the cache with 200k prompts.
- **Document comprehension based on TriviaQA [32].** Because TriviaQA is initially used for training, it has an almost uniform number of visits per document. Therefore, we introduce skewness by following Zipf distributions similar to prior studies on data caching [6, 9, 22, 28, 29]. We evaluate Zipf with low and high skewness levels: α=0.4 (10 % of documents are accessed by ~25 % of prompts) and α=0.7 (10 % of documents are accessed by ~50 % of prompts). Prompts are also generated under Poisson distributions. As the context is larger than multi-turn conversations, we initialize the cache with 50k prompts.

Request rate. Prior work has shown that the request rate to LLM services varies in a day [58, 70]. To reflect realistic load dynamics, we use the Azure LLM trace [3] to simulate prompt arrivals



Multi-turn conversation. Document comp. ($\alpha = 0.4$). Document comp. ($\alpha = 0.7$).

(a) Carbon emissions of Llama-3 70B.



Multi-turn conversation. Document comp. ($\alpha = 0.4$). Document comp. ($\alpha = 0.7$).

(b) Carbon emissions of Llama-3 8B.

Fig. 12. Average carbon emissions of LLM tasks.

at corresponding request rates. We downscale the request rate of the Azure trace to match our platform's capacity, ensuring that the peak rate is still within our system's sustainable throughput. The load predictor in Section 5.3 forecasts future request rates using past and current rate data.

Carbon intensity. We evaluate four grids, FR, FI, ES, and CISO, and predict their CI using EnsembleCI [76], which is trained with 18 months of data and predicts CI for July 6, 2022. The training data and groundtruth CI come from the CarbonCast dataset [49]. We use the CI on this date for the main performance and carbon evaluation, where we incorporate the variable CI traces with the variable request rate to simulate a realistic scenario. We also use the average CI of the ES grid for sensitivity and ablation studies.

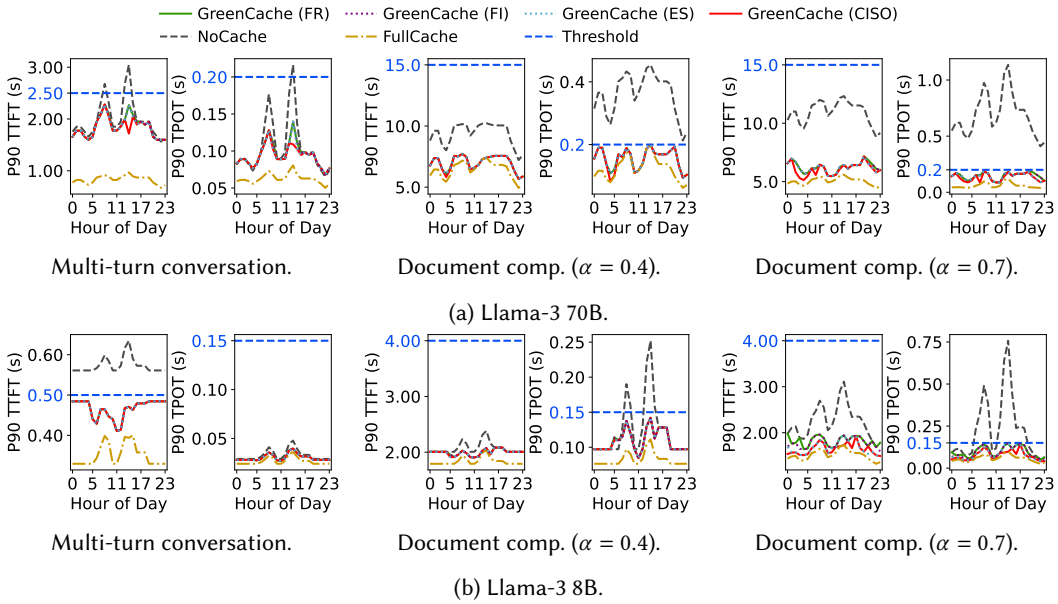
SLOs. We define SLOs for both TTFT and TPOT. For multi-turn conversations, we set TTFT and TPOT SLOs to 2.5 s and 0.2 s for the 70B model, and 0.5 s and 0.15 s for the 8B model. For document comprehension, where inputs are longer and latency is less time-critical, we relax the TTFT thresholds to 15 s for the 70B model and 2.5 s for the 8B model. These SLOs are aligned with prior work [2, 81] and benchmarking results from LLMPerf [43]. We focus on an SLO attainment of 90 %, requiring at least 90 % requests to meet the timing constraints.

Comparison points. We evaluate the following system design points: (1) **No Cache**: Non-caching baseline with vLLM and continuous batching. (2) **Full Cache**: Use the maximum cache sizes as described in the experiment setup. (3) **GreenCache**: The carbon-aware caching system in this work. The maximum cache size is the same as Full Cache.

6.2 Carbon Emission and SLO Attainment

We evaluate the carbon emissions and SLO attainment of GreenCache using dynamic CI traces of four grids, where each grid follows an Azure request rate trace.

Carbon Emissions. First, we present the overall carbon emissions when serving Llama-3 70B and 8B models, as Figures 12a and 12b show, respectively. Compared to Full Cache, GreenCache



(a) Llama-3 70B.

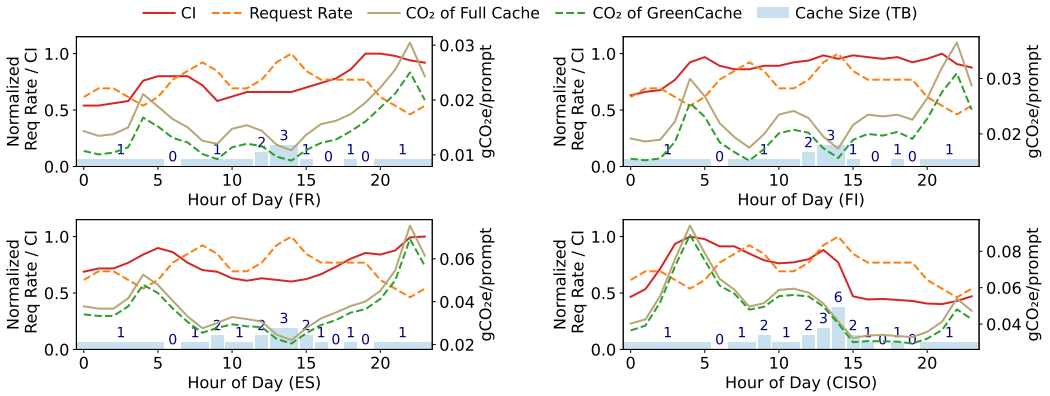
(b) Llama-3 8B.

Fig. 13. SLO attainment timelines of LLM tasks.

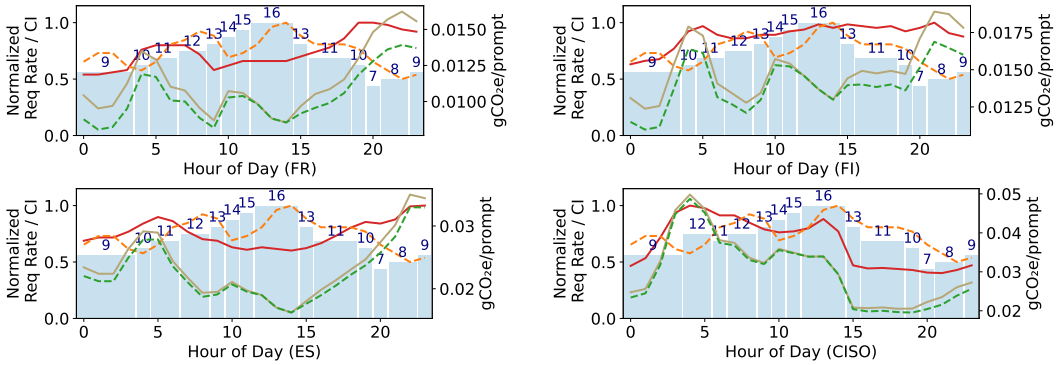
achieves average 12.6 %, 9.4 %, and 5.6 % lower carbon in multi-turn conversation and document comprehension with two skewness levels, respectively, when serving Llama-3 70B. In comparison, when serving Llama-3 8B, the carbon emission reductions are slightly lower – 10.8 %, 7.6 %, and 9.7 %, respectively, as the 8B model is lightweight and requires a smaller cache. The carbon emission savings come from a reduced cache usage over Full Cache, as the cache size labels indicate. We also observe that GreenCache achieves higher savings in low-CI grids like FR and FI, 14.4–20.3 %, as embodied carbon is more prominent in these grids. The skewness of the document compression dataset also significantly changes the carbon savings and cache sizes. The high-skewness case ($\alpha = 0.7$) requires a smaller cache size compared to the low-skewness case ($\alpha = 0.4$) as fewer prompts are frequently used. Thus, the carbon emission savings from GreenCache in the high-skewness case are higher. We notice that although No Cache sometimes achieves lower carbon emissions than GreenCache. However, this option is not acceptable due to the violation of SLOs, which will be discussed next.

SLO Attainment. We evaluate GreenCache's SLO attainment against baselines in the four grids. Figures 13a and 13b present the P90 TTFT and TPOT through a day of both models, and compare them against the thresholds as specified by the SLOs. The P90 latency staying below the SLO-specified thresholds indicates at least 90 % SLO attainment. Among all scenarios, GreenCache only exhibits slightly higher P90 latency than Full Cache, staying below both TTFT and TPOT thresholds as specified by the SLOs, indicating over 90 % SLO attainment. In contrast, No Cache's P90 latency exceeds the SLO constraints frequently, which is not considered a viable solution.

Timelines. Finally, we showcase timelines that demonstrate the dynamics of cache size and per-prompt carbon emissions under real-time CI and request rate for Llama-3 70B. Figures 14a and 14b show the timelines for the multi-turn conversation and document comprehension (with skewness $\alpha = 0.4$) tasks, respectively. The y-axis presents the CI and request rate normalized by their highest values. We do not include the No Cache baseline in the timeline as it fails to meet SLO.



(a) Multi-turn conversation with Llama-3 70B.



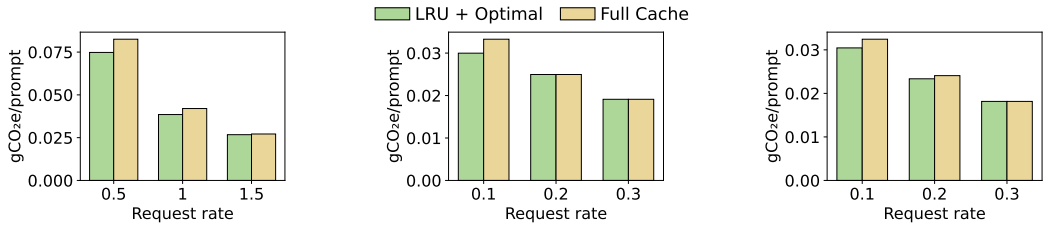
(b) Document comprehension ($\alpha = 0.4$) with Llama-3 70B.

Fig. 14. Timelines of carbon emissions under variable CI and rate.

Throughout the day, GreenCache reduces carbon emissions by 6.9–20.4 % and 3.2–16.2 % over Full Cache in multi-turn conversation and document comprehension, respectively. Among the 4 grids, FR demonstrates the most carbon reduction due to its low CI, amplifying the impact of reducing embodied carbon; GreenCache reduces carbon emissions by an average of 15.1 % and up to 25.3 % reduction in multi-turn conversation, as GreenCache utilizes 15 TB less SSD than Full Cache (12 AM of the day in Figure 14a). Specifically, cache size is more sensitive to the request rate under low CI, as a larger cache is needed to meet the SLO attainment goal. In contrast, under high CI, the cache sizes are generally larger, as caching is effective at reducing operational carbon. For example, in Figure 14a, FR, FI, and ES grids have similarly small cache sizes, while CISO has more variation in cache sizes. In Figure 14b, the difference is less prominent as this task has longer contexts and requires larger caches. For the same reason, GreenCache saves less carbon when the load is higher because a larger cache is needed (up to the full 16 TB of cache).

6.3 Ablation Studies

In this section, we first study benefits from the adaptive caching and then compare LCS with other cache replacement policies.



(a) Multi-turn conversation. (b) Document comp. (α=0.4). (c) Document comp. (α=0.7).

Fig. 15. Ablation study on adaptive caching (Llama-3 70B).

Table 3. Hit rate (Llama-3 70B).

Cache Size (TB)	ShareGPT			TriviaQA,α=0.4			TriviaQA,α=0.7		
	FIFO	LRU	LCS	FIFO	LRU	LCS	FIFO	LRU	LCS
1	0.05	0.05	**0.08**	0.05	0.05	**0.06**	0.10	0.11	**0.19**
2	0.11	0.12	**0.17**	0.08	0.08	**0.09**	0.16	0.19	**0.25**
4	0.19	0.21	**0.28**	0.12	0.12	**0.14**	0.23	0.26	**0.35**
8	0.34	0.40	**0.47**	0.19	0.20	**0.22**	0.37	0.38	**0.44**
16	0.52	0.69	**0.71**	0.31	0.32	**0.33**	0.47	**0.52**	**0.52**

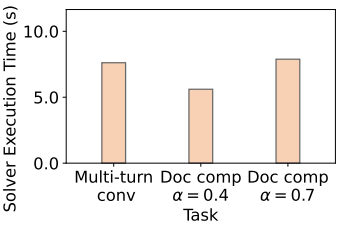


Fig. 16. Constraint solver execution time for every cache resize.

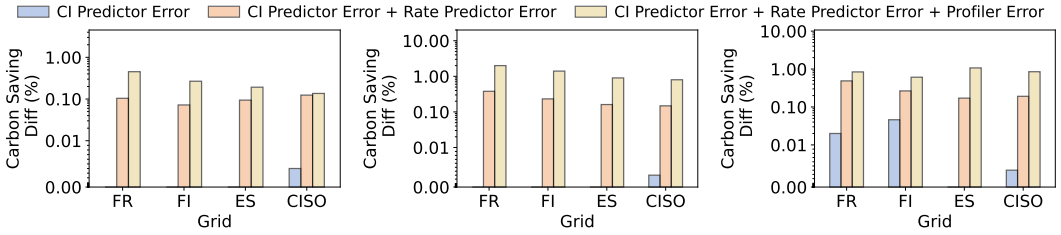
6.3.1 Adaptive Caching Analysis. We first conduct an experiment that integrates GreenCache's adaptive caching technique into LMCache [45] while using its original LRU policy (i.e., LRU + Optimal). Figure 15 shows the carbon reduction under different request rates. For this analysis, we use the average carbon intensity of the ES grid, which is 124 gCO_2e/kWh. Compared to Full Cache, GreenCache's adaptive caching can bring up to 10.3 % carbon savings in multi-turn conversation and 6.6–9.9 % reduction in document comprehension. As the request rate increases, the benefit decreases because GreenCache adapts to larger cache sizes to meet the SLO attainment goal.

6.3.2 Replacement Policy Comparison. We then compare the hit rates of the following replacement policies: (1) **First In, First Out (FIFO)** replaces blocks in the order in which they were added. (2) **Least Recently Used (LRU)** replaces the least recently used cache block. It is the default policy in LMCache [45]. (3) **Least Carbon Savings (LCS)** is the replacement policy in this work that replaces the entry that brings the least carbon emission savings (details in Section 5.5).

We evaluate cache sizes ranging from 1 to 16 TB with the Llama-3 70B model. We define *cache hit rate* as the number of tokens reused from the cache over the total number of input tokens, as shown in Table 3. In the vast majority of cases, our replacement policy LCS outperforms other replacement policies. Although LRU indicates similar performance as LCS with 16TB cache size, LCS demonstrates up to 9 % higher hit rate than LRU with smaller cache sizes. We also notice that, in the document comprehension task, higher skewness ($\alpha = 0.7$) leads to better cache hit rates in all policies, compared to low skewness ($\alpha = 0.4$).

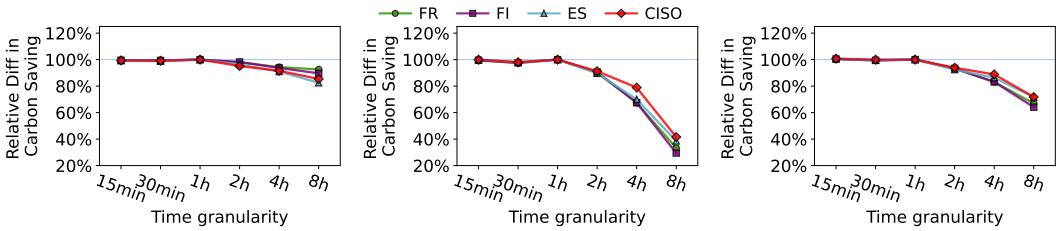
6.4 Constraint Solver Overhead

We evaluate the execution time of the constraint solver when making each decision, as shown in Figure 16. The average latency of making a cache resizing decision is as low as 7.03 s. This is a low overhead given that these serving workloads are long-running.



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 17. Impact of prediction and profile inaccuracies (Llama-3 70B).



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 18. Impact of variable cache resizing intervals (Llama-3 70B). A higher value indicates more savings.

6.5 Predictors and Profiler Errors

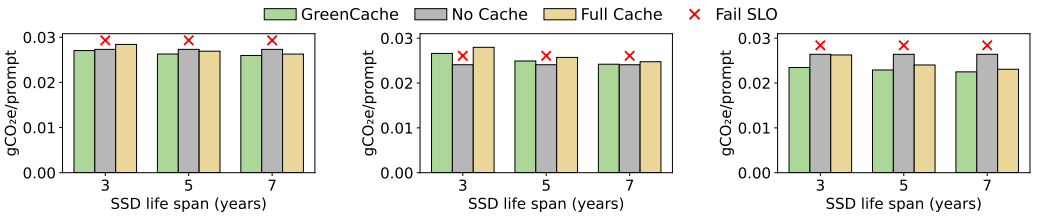
GreenCache decisions can be affected by three sources of errors, CI predictor, load predictor, and profiler as discussed section 5.4. We first calculate the Mean Average Percentage Error (MAPE) of both predictors by comparing the predictions with the groundtruth. The load predictor has an MAPE of 4.3 %, and the CI predictor has MAPE values of 12.7 %, 15.3 %, 11.3 %, and 6.8 % for FR, FI, ES, and CISO grids, respectively. Then, we calculate the distribution differences between the profiling data and the evaluation data. In multi-turn conversation and document comprehension tasks, the median context length difference is 5.78 % and 1.13 %, respectively.

We further evaluate the impact on carbon emission savings by comparing the carbon emissions of GreenCache under errors with an ideal scenario that uses the groundtruth. Figure 17 shows the reduction of carbon emission savings due to errors. To make small values visible, we plot the values on a log scale. CI prediction errors only reduce 0.0064 % of carbon savings on average compared to the ideal scenario. When load prediction errors are included, the reduction is increased to an average of 0.20 %. Although the load predictor has lower MAPEs than CI, it directly affects the choice of cache size and thus has a higher impact on carbon emissions. Finally, the errors from the profiler increase the carbon savings reduction to an average of 0.79 %. The profiler errors have a relatively higher impact in low-CI grids (FR and FI), as embodied carbon has a higher weight in these grids. We conclude that these errors have an overall low impact on carbon emission savings.

6.6 Sensitivity Studies

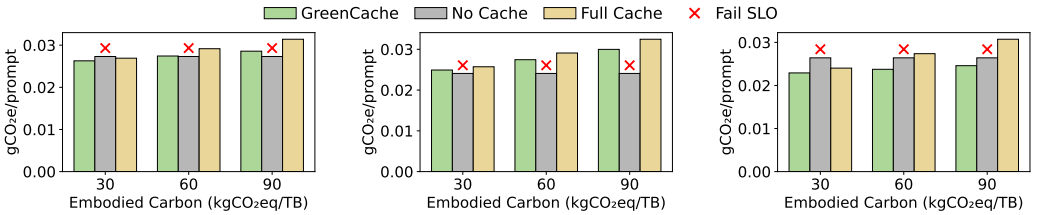
In this section, we vary the decision-making frequency, SSD lifespan, and SSD embodied carbon to analyze their impact on carbon emissions.

6.6.1 Cache Resizing Interval. We evaluate the carbon emission savings of Llama-3 70B over full cache under variable resizing intervals, in addition to the default 1-hour interval. Figure 18 shows



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 19. Variable SSD lifespan (Llama-3 70B, ES grid).



(a) Multi-turn conversation. (b) Document comp. ($\alpha = 0.4$). (c) Document comp. ($\alpha = 0.7$).

Fig. 20. Variable SSD embodied carbon (Llama-3 70B, ES grid).

the relative difference compared to the default 1-hour interval. We also adjust the CI and rate predictors to match the granularity. Because the CI data has a minimum granularity of 1 hour due to the dataset, we use the fixed hourly CI value instead when the interval is within 1 hour. GreenCache sets a sufficiently large cache size during the whole interval to ensure the SLO attainment goal. Therefore, the carbon emission savings are significantly reduced under longer intervals across all grids. Among the tasks, the difference is more significant in the document comprehension that requires larger caches, especially under a lower skewness of $\alpha = 0.4$.

6.6.2 SSD Lifespan. Prior studies have found that the lifespans of SSDs vary by manufacturer and usage. In this experiment, we study the carbon emission savings from GreenCache by varying the SSD lifetime from 3 to 7 years [1, 47, 48, 51]. We use a fixed rate of 1.5 prompts/s for the multi-turn conversation task and 0.2 prompts/s for document comprehension. Like the ablation studies, we also use the average carbon intensity of the ES grid. Figure 19 indicates that a shorter, 3-year SSD lifetime leads to more carbon savings from GreenCache, with up to 11.9 %. Extending SSD lifetime reduces amortized embodied carbon, lowering the savings from resizing the cache.

6.6.3 SSD Embodied Carbon. In this work, we model SSD embodied carbon using ACT [26]. SSD embodied carbon varies by factors such as the manufacturing technology, fab location, and account scope. Recent studies have reported higher embodied carbon of SSDs [7, 71]. Therefore, we evaluate a range of embodied carbon emissions of SSDs, from 30 to 90kgCO_2e/TB. Same as the previous sensitivity study, we evaluate the two tasks with fixed rates of 1.5 and 0.2 prompts/s, respectively. We also follow the CI of the ES grid. Figure 20 demonstrates that with the embodied carbon of SSD increasing, the carbon benefits from GreenCache also increase as embodied carbon has a higher weight in the total emissions. GreenCache can reduce more embodied carbon by shrinking the cache size. Overall, GreenCache saves up to 25 % carbon if the SSD's embodied carbon is 90 kgCO_2e/TB.

7 Discussion

In this section, we discuss the assumptions of embodied carbon accounting, potential improvement from more advanced load and CI predictors, and implications on MoE models.

Embodied Carbon Accounting. Consistent with prior work [27, 39, 67, 68, 73], we consider both embodied and operational carbon emissions in cloud environments, emphasizing that embodied carbon is a critical component that should be included. Different from the assumptions in Sunk Carbon Fallacy [5], which treat unselected resources as idle and their embodied carbon as a fixed sunk cost, this work assumes a cloud scenario in which unused resources are promptly utilized by other workloads. Consequently, we model embodied carbon as continuously amortized across workloads over time and explicitly account for it in resource provisioning and management. In particular, for storage that this work focuses on, GreenCache dynamically resizes storage, and thus the embodied carbon of storage (i.e., SSD) is also attributed based on the actual storage provision.

Load Prediction. We design a lightweight load predictor, as discussed in Section 5.3. The main limitation of the request rate study is the lack of real-world LLM serving traces, as the publicly available Azure trace only covers 1 week. With more trace data, it is possible to achieve higher prediction accuracy, like prior works in other cloud computing domains [35, 79, 80], which can lead to even better carbon emission savings.

Carbon Intensity Forecast. GreenCache uses a recent, state-of-the-art CI prediction model, EnsembleCI [76]. CI prediction is a well-known problem in sustainable computing [49, 50, 76]. Like the load predictor, the CI predictor is an easily replaceable module in the GreenCache framework. With a more accurate prediction model, we expect more accurate cache allocation.

Implications on MoE models. Mixture-of-Expert (MoE) models are widely deployed [13, 72]. Compared to dense LLMs, MoE models incorporate multiple FFNs as "experts" and activate only a subset of them, leading to lower computation overhead. On the other hand, the KV cache storage is not reduced. Therefore, MoE models lower operational carbon emissions but amplify the significance of embodied carbon, making the optimizations provided by GreenCache more impactful.

8 Related Work

In this section, we discuss caching for LLM systems and other carbon mitigation strategies.

Caching for LLM. Storing KV caches among different requests is a common optimization [20, 21, 42]. For example, CachedAttention [20] proposes to reuse the KV cache among multi-turn conversations with TBs of caches. CacheGen [42] reduces the network overhead by compressing the transferred KV caches when the SSD is deployed remotely. HCache [21] restores historical LLM states with intermediate activations and manages the chunk-wise storage to alleviate the overhead from I/O or recomputing. Instead of the precise matching in tokens, prior works also propose semantic caching. For example, GPTCache [4] retrieves the cache by its semantics and returns corresponding responses. Unlike these studies that only target performance, GreenCache achieves a tradeoff between performance and carbon emissions. The existing caching solutions can also be integrated into GreenCache framework to reduce their total carbon emissions.

Carbon mitigation for LLM. Prior works have proposed various solutions to alleviate carbon emissions of LLMs [19, 63, 67, 68, 70]. For example, DynamoLLM [70] schedules and manages requests on various GPUs under diverse frequency settings to minimize operational carbon while satisfying SLOs. LLMCarbon [19] builds a carbon model to predict the emissions due to LLM training and inference. GreenLLM [67, 68] notices the heterogeneous demand in LLM and proposes to use different types of GPUs to lower both embodied and operational carbon. From Words to Watts [63] benchmarks LLM on various settings to show the energy and performance efficiency. EcoServe [39] analyzes the proportion of the operational and embodied carbon, and schedules

the LLM applications with resource provision. These works provide substantial carbon emission savings from the computation side but overlook the importance of the embodied carbon from storage, which is GreenCache's main angle.

9 Conclusions

The wide use of large language models (LLMs) leads to high carbon emissions. While most LLM sustainability studies focus on compute-related emissions, we find that storage for saving the historical KV cache is another significant contributor. Although caching improves performance and reduces operational emissions, it also introduces substantial embodied carbon due to the high-speed, high-capacity SSDs. To address this tradeoff, we introduce GreenCache, a carbon-aware caching framework that dynamically profiles LLM tasks and uses an ILP-based optimizer to reconfigure cache size, balancing performance and carbon efficiency while meeting the SLO attainment goal.

Acknowledgement

We thank the anonymous reviewers and the shepherd for their valuable feedback on this work, and Shuncheng Jie and Dengwang Tang for proofreading. We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC) RGPIN-2023-03478. We also acknowledge the support of the U.S. National Science Foundation (NSF) CCF-2413870.

References

[1] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Kiwan Maeng, Udit Gupta, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. 2023. Carbon Explorer: A Holistic Framework for Designing Carbon Aware Datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 118–132. doi:10.1145/3575693.3575754

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, Santa Clara, CA, USA, 117–134. https://www.usenix.org/conference/osdi24/presentation/agrawal

[3] Azure. 2024. Azure LLM inference trace 2024. https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2024.md.

[4] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*. Association for Computational Linguistics, Singapore, 212–218. doi:10.18653/v1/2023.nlposs-1.24

[5] Noman Bashir, Varun Gohil, Anagha Belavadi Subramanya, Mohammad Shahrad, David Irwin, Elsa Olivetti, and Christina Delimitrou. 2024. The Sunk Carbon Fallacy: Rethinking Carbon Footprint Metrics for Effective Carbon-Aware Scheduling. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC)*. Association for Computing Machinery, New York, NY, USA, 542–551. doi:10.1145/3698038.3698542

[6] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. 2020. The CacheLib Caching Engine: Design and Experiences at Scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Virtual, 753–768. https://www.usenix.org/conference/osdi20/presentation/berg

[7] Anvita Bhagavathula, Leo Han, and Udit Gupta. 2024. Understanding the Implications of Uncertainty in Embodied Carbon Models for Sustainable Computing. In *Workshop on Sustainable Computer Systems (HotCarbon)*. ACM, New York, NY, USA, 1–7.

[8] Hendrik Borghorst. 2018. rapl-read-ryzen. https://github.com/djselbeck/rapl-read-ryzen.

[9] L. Breslau, Pei Cao, Li Fan, G. Phillips, and S. Shenker. 1999. Web caching and Zipf-like distributions: evidence and implications. In *IEEE INFOCOM '99. Conference on Computer Communications. Proceedings. Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies. The Future is Now (Cat. No.99CH36320)*, Vol. 1. IEEE Computer Society, New York, NY, USA, 126–134 vol.1. doi:10.1109/INFCOM.1999.749260

[10] Zhiliang Chen, Xinyuan Niu, Chuan-Sheng Foo, and Bryan Kian Hsiang Low. 2025. Broaden your SCOPE! Efficient Multi-turn Conversation Planning for LLMs with Semantic Space. In *The Thirteenth International Conference on Learning Representations (ICLR)*. OpenReview.net, Singapore. https://openreview.net/forum?id=3cgMU3TyyE

[11] Yihua Cheng, Kuntai Du, Jiayi Yao, and Junchen Jiang. 2024. Do Large Language Models Need a Content Delivery Network? *arXiv preprint arXiv:2409.13761* (2024).
[12] COIN-OR Foundation. 2005–. CBC (Coin-or branch and cut) solver. https://github.com/coin-or/Cbc. Open-source MILP solver from the COIN-OR project.
[13] Damai Dai, Chengqi Deng, Chenggang Zhao, R.x. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y.k. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 1280–1297. doi:10.18653/v1/2024.acl-long.70
[14] DeepSeek. 2025. DeepSeek. https://chat.deepseek.com/.
[15] Dell Technologies. 2019. Life Cycle Assessment of Dell R740. https://www.delltechnologies.com/asset/en-us/products/servers/technical-support/Full_LCA_Dell_R740.pdf.
[16] Yi Ding and Tianyao Shi. 2024. Sustainable LLM Serving: Environmental Implications, Challenges, and Opportunities. In *2024 IEEE 15th International Green and Sustainable Computing Conference (IGSC)*. IEEE, IEEE, Austin, TX, USA, 37–38.
[17] Hang Du, Guoshun Nan, Sicheng Zhang, Binzhu Xie, Junrui Xu, Hehe Fan, Qimei Cui, Xiaofeng Tao, and Xudong Jiang. 2024. DocMSU: A Comprehensive Benchmark for Document-Level Multimodal Sarcasm Understanding. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 16 (Mar. 2024), 17933–17941. doi:10.1609/aaai.v38i16.29748
[18] Electricity Maps. 2025. Electricity Maps. https://www.electricitymap.org/map/.
[19] Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Chukwunyere Osi, Prateek Sharma, Fan Chen, and Lei Jiang. 2024. LLMCarbon: Modeling the End-to-End Carbon Footprint of Large Language Models. In *The Twelfth International Conference on Learning Representations (ICLR)*. OpenReview.net, Vienna, Austria. https://openreview.net/forum?id=aIok3ZD9to
[20] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*. USENIX Association, Santa Clara, CA, 111–126. https://www.usenix.org/conference/atc24/presentation/gao-bin-cost
[21] Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast State Restoration in LLM Serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*. Association for Computing Machinery, New York, NY, USA, 128–143. doi:10.1145/3689031.3696072
[22] Phillipa Gill, Martin Arlitt, Zongpeng Li, and Anirban Mahanti. 2007. YouTube Traffic Characterization: A View from the Edge. In *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement (IMC)*. Association for Computing Machinery, New York, NY, USA, 15–28. doi:10.1145/1298306.1298310
[23] GitHub. 2024. copilot. https://github.com/features/copilot.
[24] Google. 2024. Gemini. https://gemini.google.com/app.
[25] Sarah Griffiths. 2020. Why your internet habits are not as clean as you think. https://www.bbc.com/future/article/20200305-why-your-internet-habits-are-not-as-clean-as-you-think.
[26] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S. Lee, David Brooks, and Carole-Jean Wu. 2022. ACT: Designing Sustainable Computer Systems with An Architectural Carbon Modeling Tool. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*. Association for Computing Machinery, New York, NY, USA, 784–799. doi:10.1145/3470496.3527408
[27] Leo Han, Jash Kakadia, Benjamin C. Lee, and Udit Gupta. 2025. Fair-CO2: Fair Attribution for Cloud Carbon Emissions. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*. Association for Computing Machinery, New York, NY, USA, 646–663. doi:10.1145/3695053.3731023
[28] Syed Hasan, Sergey Gorinsky, Constantine Dovrolis, and Ramesh K. Sitaraman. 2014. Trade-offs in optimizing the cache deployments of CDNs. In *IEEE INFOCOM 2014 - IEEE Conference on Computer Communications*. IEEE, Toronto, Canada, 460–468. doi:10.1109/INFOCOM.2014.6847969
[29] Qi Huang, Ken Birman, Robbert van Renesse, Wyatt Lloyd, Sanjeev Kumar, and Harry C. Li. 2013. An analysis of Facebook photo caching. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 167–181. doi:10.1145/2517349.2522722
[30] Hugging Face. 2023. ShareGPT_Vicuna_unfiltered.
[31] Jinwoo Jeong and Jeongseob Ahn. 2025. Accelerating LLM Serving for Multi-turn Dialogues with Efficient Resource Management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3676641.3716245

[32] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, Vancouver, Canada, 1601–1611. doi:10.18653/v1/P17-1147

[33] Zhaokang Ke, Dingyi Kang, Bo Yuan, David Du, and Bingzhe Li. 2024. Improving the Sustainability of Solid-State Drives by Prolonging Lifetime. In *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, Knoxville, TN, USA, 502–507. doi:10.1109/ISVLSI61997.2024.00096

[34] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[35] Ang Li, Xuanran Zong, Srikanth Kandula, Xiaowei Yang, and Ming Zhang. 2011. CloudProphet: Towards Application Performance Prediction in Cloud. In *Proceedings of the ACM SIGCOMM Conference* (Toronto, Ontario, Canada) *(SIGCOMM)*. Association for Computing Machinery, New York, NY, USA, 426–427. doi:10.1145/2018436.2018502

[36] Baolin Li, Rohan Basu Roy, Daniel Wang, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. 2023. Toward Sustainable HPC: Carbon Footprint Estimation and Environmental Implications of HPC Systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. Association for Computing Machinery, New York, NY, USA, Article 19, 15 pages. doi:10.1145/3581784.3607035

[37] Baolin Li, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. 2023. Clover: Toward Sustainable AI with Carbon-Aware Machine Learning Inference Service. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. Association for Computing Machinery, New York, NY, USA, Article 20, 15 pages. doi:10.1145/3581784.3607034

[38] Yueying Li, Omer Graif, and Udit Gupta. 2024. Towards Carbon-efficient LLM Life Cycle. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon)*. ACM, New York, NY, USA.

[39] Yueying Li, Zhanqiu Hu, Esha Choukse, Rodrigo Fonseca, G Edward Suh, and Udit Gupta. 2025. Ecoserve: Designing carbon-aware ai inference systems. *arXiv preprint arXiv:2502.05043* (2025).

[40] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Boston, MA, 663–679. https://www.usenix.org/conference/osdi23/presentation/li-zhouhan

[41] Shuo Liu, Kaining Ying, Hao Zhang, Yue Yang, Yuqi Lin, Tianle Zhang, Chuanhao Li, Yu Qiao, Ping Luo, Wenqi Shao, and Kaipeng Zhang. 2024. ConvBench: A Multi-Turn Conversation Evaluation Benchmark with Hierarchical Ablation Capability for Large Vision-Language Models. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., Vancouver, BC, Canada, 100734–100782. https://proceedings.neurips.cc/paper_files/paper/2024/file/b69396afc07a9ca3428d194f4db84c02-Paper-Datasets_and_Benchmarks_Track.pdf

[42] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference (SIGCOMM)*. Association for Computing Machinery, New York, NY, USA, 38–56. doi:10.1145/3651890.3672274

[43] LLMPerf. 2024. LLMPerf Leaderboard.

[44] LMCache Team. 2025. KV Cache Size Calculator. https://lmcache.ai/kv_cache_calculator.html.

[45] LMCache Team. 2025. LMCache. https://lmcache.ai/.

[46] Chuwei Luo, Yufan Shen, Zhaoqing Zhu, Qi Zheng, Zhi Yu, and Cong Yao. 2024. LayoutLLM: Layout Instruction Tuning with Large Language Models for Document Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Seattle, WA, USA, 15630–15640.

[47] Jialun Lyu, Jaylen Wang, Kali Frost, Chaojie Zhang, Celine Irvene, Esha Choukse, Rodrigo Fonseca, Ricardo Bianchini, Fiodar Kazhamiaka, and Daniel S. Berger. 2023. Myths and Misconceptions Around Reducing Carbon Embedded in Cloud Platforms. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon)*. ACM, Boston, MA, USA, Article 7, 7 pages. doi:10.1145/3604930.3605717

[48] Jialun Lyu, Marisa You, Celine Irvene, Mark Jung, Tyler Narmore, Jacob Shapiro, Luke Marshall, Savyasachi Samal, Ioannis Manousakis, Lisa Hsu, Preetha Subbarayalu, Ashish Raniwala, Brijesh Warrier, Ricardo Bianchini, Bianca Schroeder, and Daniel S. Berger. 2023. Hyrax: Fail-in-Place Server Operation in Cloud Platforms. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Boston, MA, USA, 287–304. https://www.usenix.org/conference/osdi23/presentation/lyu

[49] Diptyaroop Maji, Prashant Shenoy, and Ramesh K Sitaraman. 2023. Multi-Day Forecasting of Electric Grid Carbon Intensity Using Machine Learning. In *Proceedings of the 9th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys)*. ACM, New York, NY, USA, 19–33.

[50] Diptyaroop Maji, Ramesh K. Sitaraman, and Prashant Shenoy. 2022. DACF: Day-ahead Carbon Intensity Forecasting of Power Grids using Machine Learning. In *Proceedings of the Thirteenth ACM International Conference on Future Energy Systems (e-Energy)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3538637.3538849

[51] Sara McAllister, Fiodar Kazhamiaka, Daniel S Berger, Rodrigo Fonseca, Kali Frost, Aaron Ogus, Maneesh Sah, Ricardo Bianchini, George Amvrosiadis, Nathan Beckmann, et al. 2024. A call for research on storage emissions. *ACM SIGENERGY Energy Informatics Review* 4, 5 (2024), 67–75.

[52] Sara McAllister, Yucong "Sherry" Wang, Benjamin Berg, Daniel S. Berger, George Amvrosiadis, Nathan Beckmann, and Gregory R. Ganger. 2024. FairyWREN: A Sustainable Cache for Emerging Write-Read-Erase Flash Interfaces. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Santa Clara, CA, 745–764. https://www.usenix.org/conference/osdi24/presentation/mcallister

[53] Meta. 2024. Introducing Meta Llama 3: The most capable openly available LLM to date. https://ai.meta.com/blog/meta-llama-3/.

[54] Micron. 2025. DDR4 SDRAM memory. https://www.micron.com/products/memory/dram-components/ddr4-sdram.

[55] Sophia Nguyen, Beihao Zhou, Yi Ding, and Sihang Liu. 2024. Towards Sustainable Large Language Model Serving. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon)*. ACM, New York, NY, USA.

[56] OpenAI. 2023. ChatGPT. https://chatgpt.com/.

[57] George Ostrouchov, Don Maxwell, Rizwan A. Ashraf, Christian Engelmann, Mallikarjun Shankar, and James H. Rogers. 2020. GPU lifetimes on titan supercomputer: Survival analysis and reliability. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE/ACM, Atlanta, Georgia, USA, 41.

[58] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise improves GPU usage by splitting LLM inference phases. In *International Symposium on Computer Architecture (ISCA)*. IEEE Press, Buenos Aires, Argentina, 118–132.

[59] Cheng Peng, Xi Yang, Aokun Chen, Kaleb E. Smith, Nima PourNejatian, Anthony B. Costa, Cheryl Martin, Mona G. Flores, Ying Zhang, Tanja Magoc, Gloria Lipori, Duane A. Mitchell, Naykky S. Ospina, Mustafa M. Ahmed, William R. Hogan, Elizabeth A. Shenkman, Yi Guo, Jiang Bian, and Yonghui Wu. 2023. A study of generative large language model for medical research and healthcare. *npj Digital Medicine* 6, 1 (2023), 210. doi:10.1038/s41746-023-00958-w

[60] PuLP developers. 2025. PuLP: A Python Linear Programming API.

[61] pyNVML Developers. 2025. pyNVML. https://pypi.org/project/nvidia-ml-py/.

[62] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079 [cs.DC] https://arxiv.org/abs/2407.00079

[63] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. 2023. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. In *IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, Boston, MA, USA, 1–9. doi:10.1109/HPEC58863.2023.10363447

[64] Samsung. 2023. Samsung V-NAND SSD 990 PRO. https://download.semiconductor.samsung.com/resources/data-sheet/samsung_nvme_ssd_990_pro_datasheet_rev.2.0.pdf.

[65] Seagate. 2025. The Decarbonizing Data Report. https://www.seagate.com/ca/en/resources/decarbonizing-data-report/.

[66] ShareGPT. 2023. ShareGPT.

[67] Tianyao Shi, Yanran Wu, Sihang Liu, and Yi Ding. 2024. GreenLLM: Disaggregating Large Language Model Serving on Heterogeneous GPUs for Lower Carbon Emissions. arXiv:2412.20322 [cs.AR] https://arxiv.org/abs/2412.20322

[68] Tianyao Shi, Yanran Wu, Sihang Liu, and Yi Ding. 2025. Disaggregated Speculative Decoding for Carbon-Efficient LLM Serving. *IEEE Computer Architecture Letters* 24, 2 (2025), 369–372. doi:10.1109/LCA.2025.3630094

[69] Taylor G. Smith et al. 2017–. pmdarima: ARIMA estimators for Python. http://www.alkaline-ml.com/pmdarima

[70] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, Las Vegas, NV, USA, 1348–1362. doi:10.1109/HPCA61900.2025.00102

[71] Swamit Tannu and Prashant J. Nair. 2023. The Dirty Secret of SSDs: Embodied Carbon. *SIGENERGY Energy Inform. Rev.* 3, 3 (Oct. 2023), 4–9. doi:10.1145/3630614.3630616

[72] Gemini Team. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530 [cs.CL] https://arxiv.org/abs/2403.05530

[73] Jaylen Wang, Daniel S. Berger, Fiodar Kazhamiaka, Celine Irvene, Chaojie Zhang, Esha Choukse, Kali Frost, Rodrigo Fonseca, Brijesh Warrier, Chetan Bansal, Jonathan Stern, Ricardo Bianchini, and Akshitha Sriraman. 2025. Designing

Cloud Servers for Lower Carbon. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE Press, Buenos Aires, Argentina, 452–470. doi:10.1109/ISCA59077.2024.00041

[74] Vinnie Wong. 2023. Gen AI's Environmental Ledger: A Closer Look at the Carbon Footprint of ChatGPT. https://piktochart.com/blog/carbon-footprint-of-chatgpt/.

[75] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, et al. 2022. Sustainable AI: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems (MLSys)* (2022).

[76] Leyi Yan, Linda Wang, Sihang Liu, and Yi Ding. 2025. EnsembleCI: Ensemble Learning for Carbon Intensity Forecasting. In *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems (E-Energy)*. Association for Computing Machinery, New York, NY, USA, 208–212. doi:10.1145/3679240.3734630

[77] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*. Association for Computing Machinery, New York, NY, USA, 94–109. doi:10.1145/3689031.3696098

[78] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[79] Guoxiu Zhang, Xinyi He, and Xiaofeng Wang. 2025. PCL-RC: A Parallel Cloud Resource Load Prediction Model based on Feature Optimization. *Journal of Cloud Computing* 14, 1 (2025), 45. doi:10.1186/s13677-025-00770-9

[80] HaoFang Zhang, Jie Li, and HaoRan Yang. 2024. Cloud Computing Load Prediction Method based on CNN-BiLSTM Model under Low-Carbon Background. *Scientific Reports* 14, 1 (2024), 18004. doi:10.1038/s41598-024-68339-1

[81] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Santa Clara, CA, 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

A ILP Complexity

Theorem (NP-hardness). *The GreenCache optimization problem in Eq. (6) is NP-hard, even in a restricted setting where each time step only allows a binary cache decision (off/on), and the SLO requirement is a global ratio constraint: at least a configurable fraction ρ of all requests over the horizon satisfy the TTFT threshold and at least the same fraction ρ satisfy the TPOT threshold (where ρ is part of the input).*

Proof. We prove NP-hardness by a polynomial-time reduction from the NP-complete 0–1 Knapsack decision problem. A Knapsack instance consists of m items, where item k has weight $w_k \in \mathbb{Z}_{>0}$ and value $v_k \in \mathbb{Z}_{>0}$, a weight budget W, and a target value V. The question is whether there exists a subset $K \subseteq \{1, \ldots, m\}$ such that $\sum_{k \in K} w_k \leq W$ and $\sum_{k \in K} v_k \geq V$.

Decision version of GreenCache. Consider the decision form: given a carbon budget C, is there a cache plan such that total carbon $\leq C$ and the (global-ρ) SLO constraints hold?

Construction. Given a Knapsack instance, we construct a restricted GreenCache instance as follows.

- Set $T = m$, which maps the number of items (m) in the Knapsack problem directly to the total number of time steps (T) in the GreenCache instance. Map item k to time step $t = k$. Restrict the cache decision at each step to $S_k \in \{0, 1\}$ (cache off/on).
- The Knapsack target value V maps to the minimum number of requests that must satisfy the SLO constraints in the GreenCache instance.
- Set the request volume at time step k to be $\lambda_k := v_k$. Let $\Lambda := \sum_{k=1}^{m} \lambda_k = \sum_{k=1}^{m} v_k$. If $V > \Lambda$, the knapsack instance is trivially infeasible; in this case we output any GreenCache instance that is trivially infeasible (e.g., by setting an impossible SLO), and the reduction remains correct. Hence, assume $V \leq \Lambda$.

- Fix the workload SLO thresholds θ_{TTFT} and θ_{TPOT}. For each step k, let q_k^{TTFT} (resp. q_k^{TPOT}) denote the number of requests at step k whose TTFT (resp. TPOT) meets the corresponding threshold. We enforce that at least a fraction ρ of all requests satisfy each threshold:

$$\sum_{k=1}^{m} q_k^{\text{TTFT}} \geq \rho\Lambda, \qquad \sum_{k=1}^{m} q_k^{\text{TPOT}} \geq \rho\Lambda.$$

  We set the (input) ratio parameter to

$$\rho := \frac{V}{\Lambda} \in (0, 1].$$

  Hence $\rho\Lambda = V$, and the global-ρ constraints become $\sum_k q_k^{\text{TTFT}} \geq V$ and $\sum_k q_k^{\text{TPOT}} \geq V$.
- We choose the instance's latency functions/coefficients so that, for each step k:

(1) If $S_k = 1$, then all λ_k requests have TTFT and TPOT below the corresponding thresholds, i.e., $q_k^{\text{TTFT}} = q_k^{\text{TPOT}} = \lambda_k$.
(2) If $S_k = 0$, then all λ_k requests violate at least one of the two thresholds, so $q_k^{\text{TTFT}} = q_k^{\text{TPOT}} = 0$.

  Therefore, the number of requests satisfying each threshold at step k is $q_k^{\text{TTFT}} = q_k^{\text{TPOT}} = \lambda_k S_k$.
- Set the incremental carbon at time step k to be exactly $w_k \cdot S_k$, and set the carbon budget to $C := W$. Any carbon terms that are independent of the cache decisions (e.g., constant embodied carbons) are in the budget C without affecting NP-hardness.

Correctness. By construction, for every k we have $q_k^{\text{TTFT}} = q_k^{\text{TPOT}} = \lambda_k S_k$. Thus, the global SLO constraints become

$$\sum_{k=1}^{m} \lambda_k S_k \geq V \quad \Longleftrightarrow \quad \sum_{k=1}^{m} v_k S_k \geq V.$$

Meanwhile, the carbon budget constraint is exactly

$$\sum_{k=1}^{m} w_k S_k \leq W.$$

Therefore, there exists a feasible solution to the constructed GreenCache decision instance iff there exists a feasible knapsack subset.

Polynomial-time reduction. The construction sets $T = m$ and uses only $\{(w_k, v_k)\}_{k=1}^{m}$ and simple arithmetic. The rational parameter $\rho = V/\Lambda$ has polynomial bit-length (it can be represented by the integer pair (V, Λ)). Hence the reduction runs in polynomial time.

Therefore, the GreenCache problem is NP-hard even in this restricted special case; consequently, the general optimization problem in Equation (6) is NP-hard. □

Received October 2025; revised December 2025; accepted January 2026